



**Universidad
Europea**

PROYECTO INTEGRADOR

Técnico Superior en Desarrollo de Aplicaciones Web
Daniel Correa Villa

Tutores:

Sara Villanueva Rosa
Irene del Ricón Bello
Raquel Cerdá Losa

Introducción

El proyecto surge como una iniciativa dentro del ámbito académico para desarrollar una aplicación web destinada a gestionar las actividades y la información de un campamento infantil.

Con el objetivo de facilitar la administración y consulta de datos, este proyecto integra conocimientos de **Diseño de Interfaces, Desarrollo en Entorno Cliente, Desarrollo en Entorno Servidor y Despliegue de Aplicaciones Web.**

En su esencia, la aplicación busca ofrecer a los usuarios una plataforma intuitiva y funcional para manejar diversas actividades relacionadas con la organización del campamento. Desde la gestión de inscripciones hasta la comunicación con tutores legales y monitores, "Campamento Infantil" tiene como objetivo optimizar los procesos administrativos y mejorar la experiencia de los usuarios.

Con una estructura bien definida, una arquitectura Modelo Vista Controlador y un enfoque en la usabilidad, este proyecto busca mejorar la gestión y el disfrute de las actividades en un entorno educativo y recreativo.

Para ver el repositorio de la aplicación, acceda al siguiente link:

<https://github.com/DCV05/Proyecto-Integrador>

Palabras clave

Backend

PHP, Polaris, Gestión de rutas dinámicas, Modelo Vista Controlador, Gestión de controladores según rutas, Monolog, PHPUnit, erguncaner/table

Frontend

HTML, CSS, JavaScript, JQuery, Tailwind, Lazy Loading, UI/UX, Figma, accesibilidad, minificación, diseño responsive, validaciones en tiempo real

Bases de datos

MySQL, Modelos de Datos, Escapado de consultas, Modelos

Despliegue

Linux, Debian Bookworm, Docker, Docker-Compose, Apache, Git, GitHub, SSH Agent

Índice

<i>Introducción</i>	2
Palabras clave.....	3
Módulos formativos incluidos	5
Herramientas.....	7
<i>Objetivos</i>	11
Objetivos generales	11
Objetivos específicos del proyecto	11
<i>Desarrollo del proyecto</i>	12
Estudio del mercado	12
Modelo de datos	16
<i>Desarrollo de la aplicación</i>	20
Gestión de rutas y clases con Polaris	20
<i>Esta estructura ha permitido:</i>	20
Entorno de desarrollo con Polaris y Debian 12	21
<i>Desarrollo e implementación</i>	22
Diagrama de casos de uso.....	22
Diagrama de modelo relacional.....	23
<i>Anexos</i>	26
Configuración general de Apache para el proyecto	26
Configuración de autenticación Digest:	27

Módulos formativos incluidos

Durante el desarrollo de la aplicación ha sido necesario aplicar numerosas aptitudes aprendidas durante el Grado Superior de Desarrollo de Aplicaciones Web:

Diseño de Interfaces: Aplicación de técnicas de diseño centradas en el usuario (UX), basadas en las leyes de usabilidad aprendidas. Creación de prototipos y maquetas de las interfaces utilizando Figma funcionales, planificando la estructuración de la página web y el panel desde el inicio del proyecto.

Desarrollo en Entorno Cliente: Uso de jQuery y sus métodos para realizar validaciones en tiempo real. Estas validaciones incluyen comprobaciones inmediatas al interactuar con los campos y mensajes de error dinámicos que se presentan junto a los elementos afectados.

Desarrollo en Entorno Servidor: Implementación de la lógica de la aplicación utilizando PHP y conexión a bases de datos MySQL/MariaDB mediante la extensión **mysqli**.

Despliegue de Aplicaciones Web: Configuración y gestión del servidor Apache utilizando las configuraciones aprendidas en clase, incluyendo el uso de archivos **.htaccess** para la gestión de rutas y control de accesos. Además, se emplea Docker para crear un entorno de desarrollo, optimizando la compatibilidad y el despliegue del proyecto al servidor de producción.

Entornos de desarrollo: Uso de tecnologías como Git y GitHub para mantener un historial de cambios coherente y un repositorio con backups con el código de la aplicación en el caso de que ocurra algún tipo de error que borre el código o los recursos utilizados dentro del sistema.

Lenguaje de marcas: Uso de HTML y CSS para el desarrollo de la estructura y estética de la aplicación. Implementación de JavaScript para programar el comportamiento de la página.

Bases de datos: Desarrollo de bases de datos relacionales (MySQL) para guardar los datos de la aplicación.

Programación: Implementación de la lógica, estructura de ficheros y programación orientada a objetos inspirada en Java. Conexión con MySQL y prepared statements.

Sistemas informáticos: Uso de Linux como sistema operativo principal del proyecto. Instalación de MySQL y Apache para su uso en la aplicación. Navegación de directorios y modificación de ficheros mediante **Bash**.

Herramientas

Durante el desarrollo de la aplicación, han sido utilizadas numerosas herramientas, entre las más importantes se encuentran las siguientes

FigJam: FigJam ha sido clave en la planificación inicial del proyecto y en el desarrollo, facilitando el diseño del user persona, user story mapping y sitemap, además de la planificación de objetivos en cada entrega.

Figma: Figma se ha utilizado para definir estilos, diseñar la aplicación y crear prototipos interactivos para presentar demos a clientes antes del desarrollo.

Git: Git ha sido usado como sistema de control de versiones. Entre sus numerosas ventajas, aquellas que han dado más beneficios han sido mantener un historial de los cambios realizados durante las diferentes entregas y poder tener numerosos backups.

GitHub: GitHub ha sido utilizado como repositorio general del proyecto y poder distribuir el código durante el despliegue en producción.

Docker: Empleado como entorno de desarrollo local para garantizar que el proyecto funcione de manera uniforme en diferentes sistemas, encapsulando configuraciones y dependencias del servidor.

PHPMyAdmin: Utilizado para administrar las bases de datos, realizar consultas y gestionar tablas de una forma más sencilla.

Visual Studio Code: ha sido el editor principal por su compatibilidad con extensiones como Git y Docker.

Intelephense: Usado para autocompletar tipado de PHP y gestionar errores dentro del código de una forma más visual.

MySQL: MySQL ha sido la base de datos utilizada para almacenar todas las tablas y campos de la aplicación.

Lenguajes

Además de las herramientas que han facilitado la planificación, desarrollo y distribución del proyecto, se han utilizado una variedad de lenguajes de marcado y de programación. Los lenguajes utilizados han sido los siguientes:

PHP

PHP es el lenguaje de programación del lado del servidor utilizado para desarrollar la lógica del backend del proyecto. Su amplia variedad de librerías como cURL, Monolog y su conexión nativa con MySQL lo convierten en el lenguaje indicado para esta aplicación.

Características clave de PHP utilizadas en el proyecto:

- **Automatización de procesos:** Gestiona tareas repetitivas como la validación de usuarios y el registro de actividad en el sistema.
- **Integración con MySQL:** Permite la manipulación eficiente de la base de datos mediante mysqli, sin necesidad de composer u otras librerías externas.
- **Manejo de seguridad:** Controla permisos de acceso y protege contra inyecciones SQL y accesos no autorizados.

JavaScript

JavaScript ha sido esencial para mejorar la experiencia de usuario, permitiendo una navegación más fluida e interactiva. Su uso en combinación con jQuery ha facilitado la manipulación de elementos y la comunicación con el backend.

Características clave de JavaScript utilizadas en el proyecto:

- **Validación de formularios:** Verificar en tiempo real los datos ingresados antes de ser enviados al servidor.
- **Interfaz dinámica:** Controlar la visibilidad de elementos, mostrar mensajes de error o éxito y manejar animaciones.
- **Gestión de eventos:** Detectar y reaccionar a interacciones como clics y cambios en los campos de entrada.
- **Llamadas AJAX:** Comunicarse con el servidor sin recargar la página, permitiendo acciones como actualizar listas de usuarios o registrar asistencia.

Polaris

Polaris ha sido el entorno de desarrollo empleado para estructurar el backend de manera eficiente, proporcionando una arquitectura basada en clases que optimiza la gestión de rutas, peticiones y respuestas. Su diseño modular ha permitido una mejor organización del código y un mantenimiento más ágil.

Dentro del proyecto, Polaris ha sido fundamental para:

- **Definición de rutas sin configuraciones externas:** Permite gestionar las solicitudes a través de clases sin necesidad de modificar archivos de servidor.
- **Centralización de peticiones AJAX:** Unifica la gestión de llamadas asíncronas dentro de una misma estructura, evitando duplicaciones de código.
- **Generación de contenido dinámico:** Mediante su compilador de etiquetas, facilita la inserción de datos desde PHP dentro de las vistas sin afectar la legibilidad del código.
- **Soporte multidioma automático:** Detecta el idioma del navegador y ajusta la interfaz sin requerir configuraciones manuales.

jQuery

jQuery ha sido utilizado principalmente para simplificar la manipulación del DOM y gestionar eventos de usuario de forma eficiente.

En este proyecto, jQuery ha sido clave para:

- **Validaciones en tiempo real:** Comprobación dinámica de datos en formularios antes de su envío.
- **Gestión de eventos:** Captura y manejo de interacciones con botones, inputs y elementos interactivos.
- **Optimización de AJAX:** Simplificación de las peticiones al servidor, mejorando la fluidez de la aplicación.

HTML

HTML se ha utilizado para definir la estructura de las páginas de la aplicación, asegurando que cada componente sea semántico y accesible.

En este proyecto, HTML ha sido clave para:

- **Estructuración del contenido:** Definir la jerarquía de elementos en las páginas.
- **Integración con templates de Polaris:** Generación dinámica de HTML desde PHP mediante ViewEngine.

CSS

Para los estilos, se ha utilizado una combinación de CSS puro y TailwindCSS, lo que ha permitido una maquetación rápida y adaptable a distintos dispositivos.

En este proyecto, CSS ha sido clave para:

- **Diseño responsive:** Adaptar la interfaz a distintos tamaños de pantalla con TailwindCSS.
- **Optimización de carga:** Uso de clases utilitarias en lugar de hojas de estilo extensas.

TailwindCSS

TailwindCSS ha sido la herramienta principal para la maquetación y diseño del frontend. Ha sido esencial para el diseño responsive con sus **media queries** integradas en la librería.

En este proyecto, TailwindCSS ha sido clave para:

- **Consistencia visual:** Mantener un diseño unificado en todas las secciones de la aplicación.
- **Rápida implementación:** Aplicar estilos directamente en los elementos HTML sin necesidad de CSS adicional.
- **Optimización del rendimiento:** Generación de estilos mínimos gracias a la eliminación automática de clases no utilizadas.

Esta combinación de tecnologías ha permitido construir una aplicación web robusta, escalable y eficiente, garantizando un desarrollo ágil y una experiencia de usuario óptima.

Objetivos

Objetivos generales

El objetivo principal del proyecto es desarrollar una aplicación web moderna y funcional que permita gestionar todos los aspectos operativos de un campamento infantil, desde la inscripción de participantes hasta la organización de actividades y la comunicación con los tutores legales.

Por otro lado, se debe permitir gestionar los pagos de los diferentes participantes dentro del campamento.

Objetivos específicos del proyecto

- Diseñar una interfaz de usuario intuitiva y accesible que cumpla con las buenas prácticas de UX/UI aprendidas en **Diseño de Interfaces Web**.
- Implementar formularios con validaciones en tiempo real para asegurar la correcta captura de datos.
- Crear secciones públicas y privadas que gestionen el acceso a las funcionalidades según el tipo de usuario.
- Establecer una estructura dinámica para mostrar y modificar información en tiempo real mediante la manipulación del DOM.
- Optimizar la comunicación con el servidor utilizando peticiones asincrónicas con JSON.

Desarrollo del proyecto

Estudio del mercado

XPert-CAMPS – Campamento Multiaventura en Cantabria

 Web: <https://xpert-camps.com/>

Aspectos positivos

1. Diseño y Navegación:

La página presenta un diseño limpio y atractivo, con una paleta de colores que transmite energía y aventura, adecuada para el público objetivo.

El menú de navegación es claro y está bien estructurado, permitiendo un acceso fácil a las diferentes secciones como “Inicio”, “Campamentos”, “Región”, “Temática”, “Reservas”, “Nosotros”, “Galería”, “Blog” y “Contacto”.

2. Información Detallada de los Campamentos:

Cada campamento dispone de una página dedicada con información detallada sobre la ubicación, actividades, fechas, edades recomendadas y precios.

Se incluyen descripciones específicas de las actividades ofrecidas, como surf, multiaventura y naturaleza, lo que ayuda a los padres y participantes a comprender claramente las experiencias que se ofrecen.

3. Contenido Visual:

La utilización de imágenes de alta calidad en las secciones de los campamentos y en la galería proporciona una visión clara de las instalaciones y actividades, generando confianza en los usuarios.

4. Accesibilidad y Contacto:

La información de contacto, incluyendo números de teléfono y dirección de correo electrónico, está claramente visible en la parte superior de la página, facilitando la comunicación para consultas o reservas.

5. Blog Informativo:

La inclusión de un blog ofrece contenido adicional que puede ser útil para los padres, como consejos y noticias relacionadas con los campamentos, mejorando el compromiso y la confianza del usuario.

Aspectos a mejorar:

1. Optimización para móviles:

Aunque la página es funcional en dispositivos móviles, la experiencia podría mejorarse con una optimización más detallada para diferentes tamaños de pantalla, asegurando una navegación más fluida en teléfonos y tablets.

2. Proceso de reserva:

El proceso de reserva podría simplificarse y hacerse más intuitivo. Actualmente, algunas reservas redirigen a subdominios específicos, lo que puede generar confusión en el usuario.

3. Testimonios y opiniones:

La inclusión de testimonios de participantes anteriores o de sus padres podría añadir credibilidad y ayudar a futuros clientes a tomar decisiones informadas.

4. Preguntas frecuentes (FAQ):

Agregar una sección de preguntas frecuentes podría anticipar y resolver dudas comunes de los usuarios, mejorando la experiencia general y reduciendo la necesidad de contacto directo para consultas básicas.

5. Integración de mapas:

Incorporar mapas interactivos de las ubicaciones de los campamentos facilitaría a los padres visualizar la localización exacta y evaluar la accesibilidad desde su lugar de residencia.

Campamento Molino de Butrera

 **Web:** <https://molinodebutrera.es/campamentos-de-verano-para-ninos-en-burgos/>

Aspectos positivos

1 Diseño y Navegación:

La página presenta un diseño limpio y organizado, con una paleta de colores que evoca la naturaleza, lo cual es coherente con la temática del campamento.

El menú de navegación es intuitivo, permitiendo un acceso fácil a secciones como “Inicio”, “Instalaciones”, “Actividades”, “Campamentos” y “Contacto”.

2. Información Detallada de los Campamentos:

Se ofrece una descripción completa de las actividades disponibles, incluyendo canoas, bicicletas de montaña, tiro con arco, escalada y excursiones por entornos naturales.

La sección de “Instalaciones y Alojamiento” detalla las opciones de hospedaje, como el antiguo molino y las cabañas de madera, proporcionando una visión clara de las comodidades ofrecidas.

6. Contenido Visual:

La inclusión de imágenes de alta calidad de las instalaciones y actividades ayuda a los usuarios a visualizar el entorno y las experiencias que ofrece el campamento.

4. Preguntas frecuentes (FAQ)

La sección de FAQ aborda preguntas comunes, como las edades adecuadas para el campamento y el tipo de actividades realizadas, lo que facilita a los padres obtener información rápidamente.

Aspectos a mejorar:

3. Optimización para móviles:

Aunque la página es funcional en dispositivos móviles, la experiencia podría mejorarse con una optimización más detallada para diferentes tamaños de pantalla, asegurando una navegación más fluida en teléfonos y tablets.

4. Proceso de Reserva:

El proceso de reserva podría simplificarse y hacerse más intuitivo. Actualmente, algunas reservas redirigen a subdominios específicos, lo que puede generar confusión en el usuario.

Modelo de datos

El **sistema de gestión del campamento infantil** se organiza en una **base de datos relacional**, estructurada en torno a entidades interconectadas que facilitan la administración de usuarios, actividades, inscripciones, asistencia y pagos.

Usuarios y Participantes (users, user_details, participants): Gestiona la información de los usuarios (tutores, monitores, administradores) y los niños inscritos en el campamento.

USERS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
user_id	INT	PK	1,1
user_id2	VARCHAR(32)	UNIQUE	1,1
user_email	VARCHAR(124)	NOT NULL	1,1
user_password	VARCHAR(256)	NOT NULL	1,1
role	INT	NOT NULL	1,1
enabled	TINYINT	NOT NULL	1,1

USER_DETAILS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
detail_id	INT	PK	1,1
detail_id2	VARCHAR(32)	UNIQUE	1,1
user_id	INT	FK -> Users(user_id)	N,1
user_name	VARCHAR(256)	NOT NULL	1,1
user_email	VARCHAR(124)	NOT NULL	1,1
user_relationship	VARCHAR(128)	NOT NULL	1,1
user_birth_date	DATE	NOT NULL	1,1
user_dni	VARCHAR(9)	NOT NULL	1,1
user_phone_number	VARCHAR(16)	NOT NULL	1,1

PARTICIPANTS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
participant_id	INT	PK	1,1
participant_id2	VARCHAR(32)	UNIQUE	1,1
user_id	INT	FK -> Users(user_id)	N,1
participant_name	VARCHAR(100)	NOT NULL	1,1
participant_birth_date	DATE	NOT NULL	1,1
participant_address	VARCHAR(128)	NOT NULL	1,1
participant_allergies	TEXT	NULLABLE	1,1
participant_special_needs	TEXT	NULLABLE	1,1
participant_medical_treatment	TEXT	NULLABLE	1,1

Actividades y monitores (activities, activities_monitors, activities_participants):

Administra las actividades del campamento, asignando monitores responsables y gestionando la participación de los niños en cada una.

ACTIVITIES

Nombre del campo	Tipo de dato	Clave	Cardinalidad
activity_id	INT	PK	1,1
activity_id2	VARCHAR(32)	UNIQUE	1,1
activity_name	VARCHAR(100)	NOT NULL	1,1
activity_description	TEXT	NOT NULL	1,1
activity_time	DATETIME	NOT NULL	1,1

ACTIVITIES_MONITORS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
relation_id	INT	PK	1,1
activity_id	INT	FK -> Activities(activity_id)	N,1
monitor_id	INT	FK -> Users(user_id)	N,1

ACTIVITIES_PARTICIPANTS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
relation_id	INT	PK	1,1
activity_id	INT	FK -> Activities(activity_id)	N,1
participant_id	INT	FK -> Participants(participant_id)	N,1

Calendario (schedule): Registra los períodos de asistencia de cada participante, estableciendo fechas de inicio y finalización en el campamento.

SCHEDULE

Nombre del campo	Tipo de dato	Clave	Cardinalidad
schedule_id	INT	PK	1,1
schedule_id2	VARCHAR(32)	UNIQUE	1,1
participant_id	INT	FK -> Participants(participant_id)	N,1
start_day	DATE	NOT NULL	1,1
end_day	DATE	NOT NULL	1,1

Pagos y finanzas (payments): Gestiona los pagos realizados por los tutores, registrando montos, fechas y estados de las transacciones.

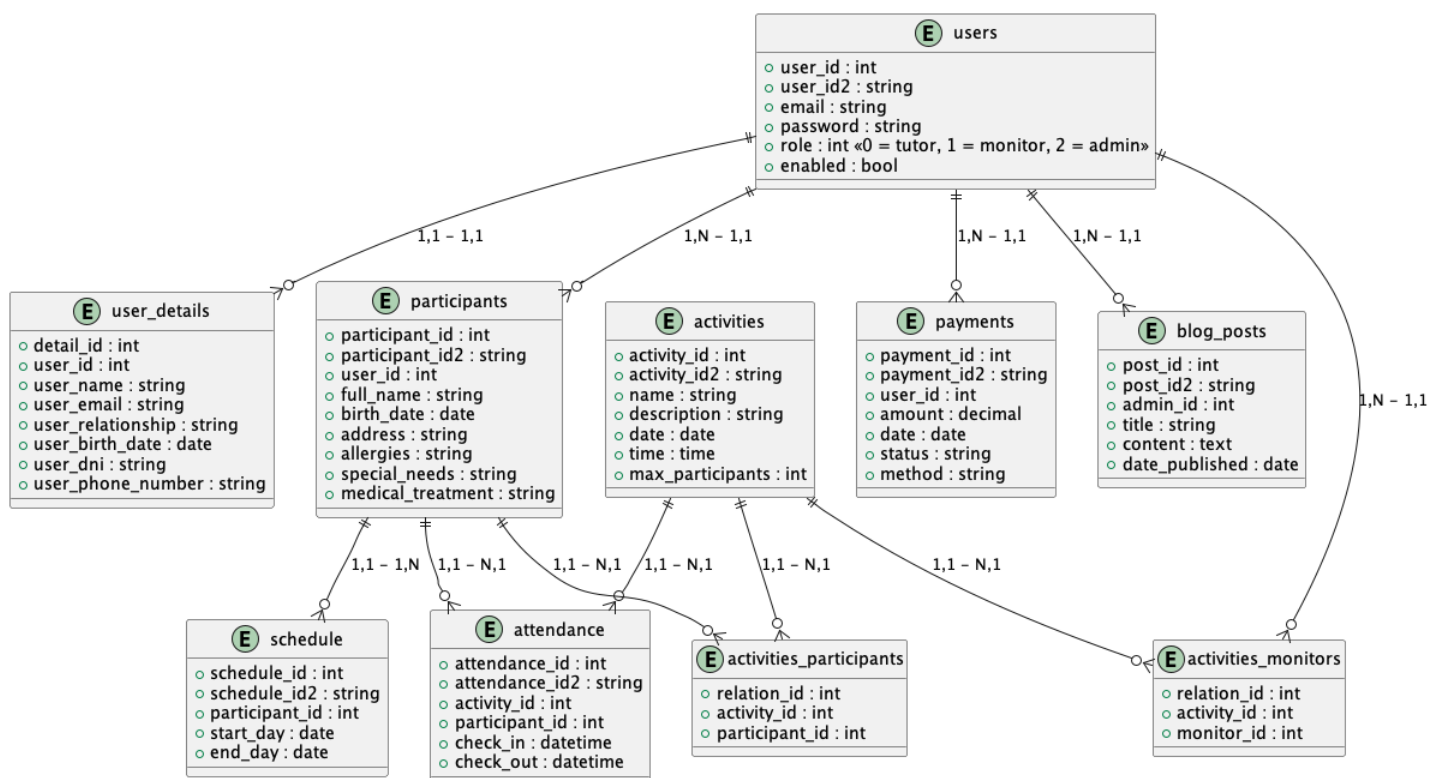
PAYMENTS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
payment_id	INT	PK	1,1
payment_id2	VARCHAR(32)	UNIQUE	1,1
user_id	INT	FK -> Users(user_id)	N,1
status	VARCHAR(32)	NOT NULL	1,1
amount	FLOAT	NOT NULL	1,1
payment_date	DATE	NOT NULL	1,1

Gestión de contenidos (blog_posts): Permite la publicación de información relevante sobre el campamento, facilitando la comunicación con los usuarios.

BLOG_POSTS

Nombre del campo	Tipo de dato	Clave	Cardinalidad
post_id	INT	PK	1,1
post_id2	VARCHAR(32)	UNIQUE	1,1
user_id	INT	FK -> Users(user_id)	N,1
post_title	VARCHAR(100)	NOT NULL	1,1
post_description	VARCHAR(256)	NOT NULL	1,1
insert_date	DATETIME	NOT NULL	1,1



Desarrollo de la aplicación

El desarrollo del sistema se ha basado en Polaris como la arquitectura principal, aprovechando su estructura basada en clases y su gestión centralizada de rutas y peticiones AJAX. Esto ha permitido una implementación más organizada y flexible, evitando la creación de múltiples archivos PHP para cada solicitud y simplificando la conexión entre el backend y el frontend.

Gestión de rutas y clases con Polaris

Desde el inicio del desarrollo, se definió una estrategia para manejar las peticiones mediante un único enrutador (`index.php`), que gestiona solicitudes GET, POST y FILES. En lugar de depender de múltiples archivos PHP dispersos, Polaris permite que todas las peticiones sean dirigidas a una clase específica según el tipo de datos recibido.

El sistema funciona de la siguiente manera:

- **Recepción de la solicitud:** Al recibir una llamada AJAX, el enrutador analiza el tipo de petición.
- **Instanciación de la clase correspondiente:** Dependiendo de la solicitud, Polaris crea una instancia de la clase encargada de procesarla.
- **Ejecución del método solicitado:** La clase ejecuta el método calculado para procesar la petición.
- **Conexión con HTML y JavaScript:** Cada clase está vinculada con una vista específica y su correspondiente archivo JavaScript, asegurando una integración clara entre backend y frontend.

Esta estructura ha permitido:

- Reducir la cantidad de archivos PHP innecesarios, centralizando la lógica del backend en una sola clase por página.
- Facilitar la gestión de AJAX, ya que todas las peticiones se procesan desde un único punto de entrada.
- Mantener un código más limpio y estructurado, mejorando la escalabilidad y el mantenimiento.

Entorno de desarrollo con Polaris y Debian 12

Uno de los mayores desafíos durante el desarrollo fue la compatibilidad con herramientas tradicionales como XAMPP, que presentaba errores en la configuración del entorno debido a incompatibilidades con el sistema. Para evitar estos problemas, se optó por una configuración más estable y flexible como Docker usando imágenes basadas en Debian 12 con PHP 8.2.

Ventajas de esta configuración:

- **Simulación de un servidor:** Docker permite ejecutar el proyecto en un entorno similar al que se utilizará en producción, evitando problemas al momento del despliegue.
- **Gestión eficiente de dependencias:** Se han utilizado herramientas como **composer** para instalar y administrar librerías sin afectar el sistema operativo de la máquina host.
- **Mayor estabilidad y control:** A diferencia de XAMPP, esta configuración ha evitado conflictos de versiones y errores inesperados durante el desarrollo.

Desarrollo e implementación

Para facilitar el desarrollo de la aplicación, se han creado varios diagramas:

Diagrama de casos de uso

El diagrama de casos de uso ha sido desarrollado para visualizar las diferentes acciones que puede realizar cada actor según su rol dentro de la aplicación, así como la forma en que cada rol interactúa con el sistema.

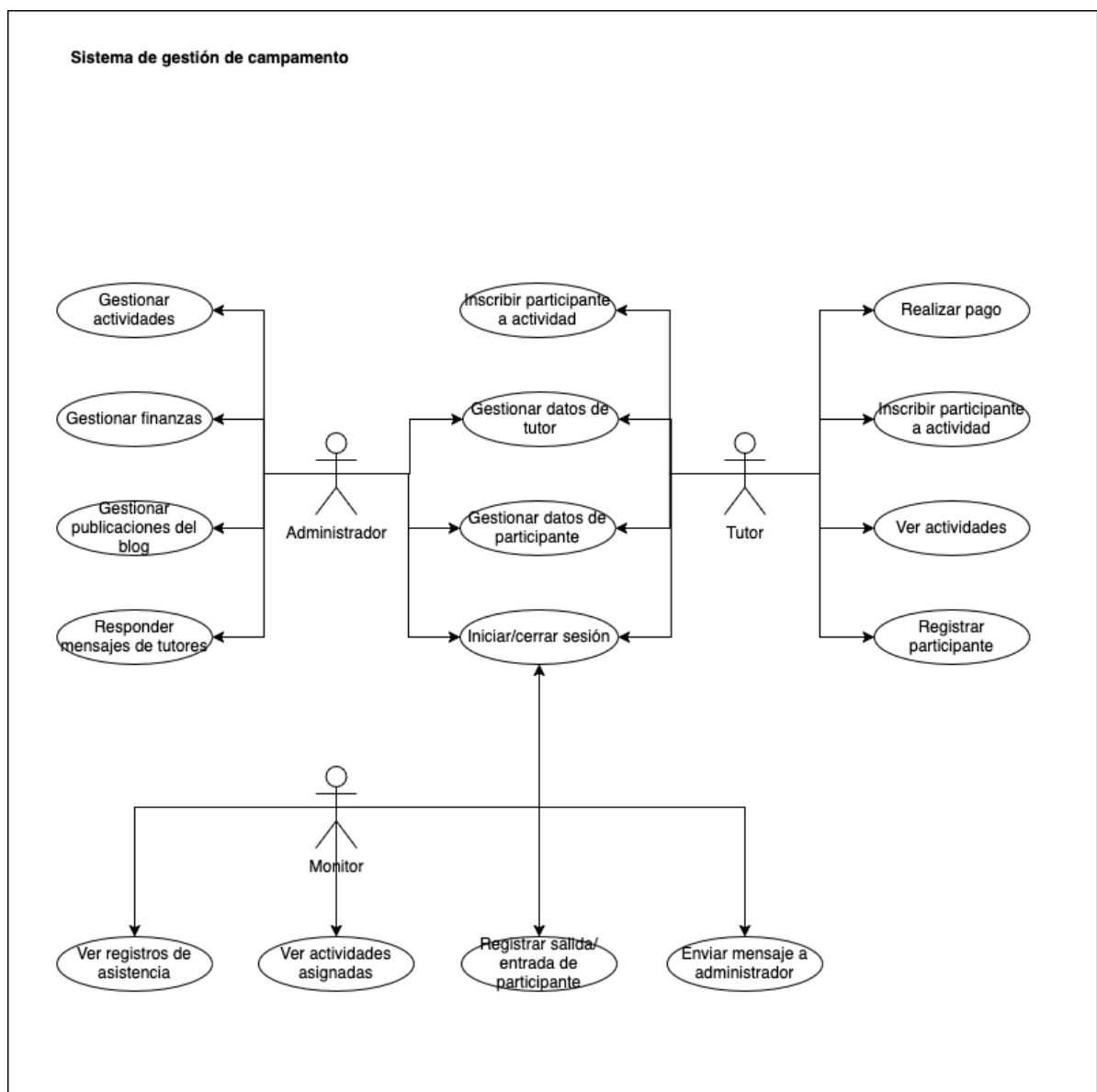


Diagrama de modelo relacional

El **modelo relacional** ha sido desarrollado para definir las tablas de la base de datos, sus columnas, tipos de datos y relaciones mediante claves primarias y foráneas.

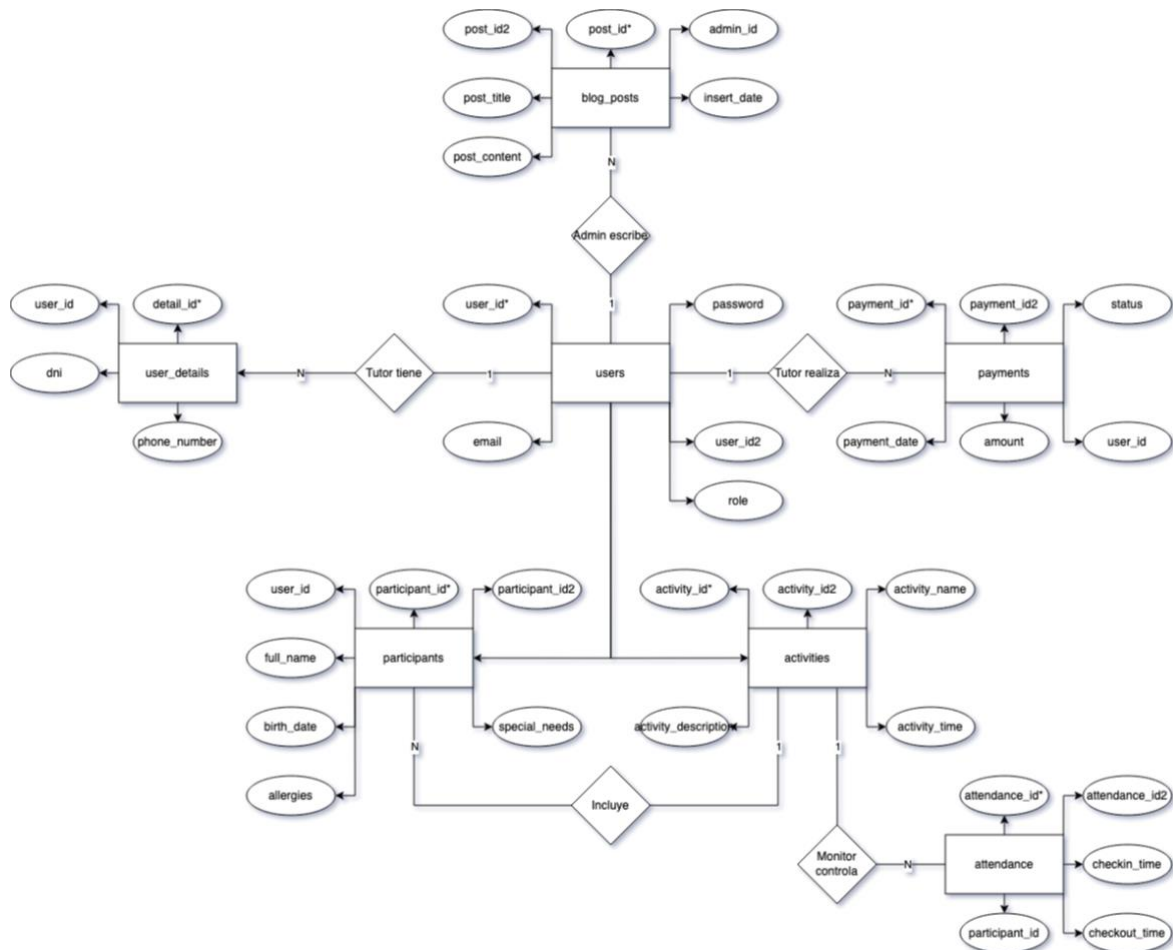
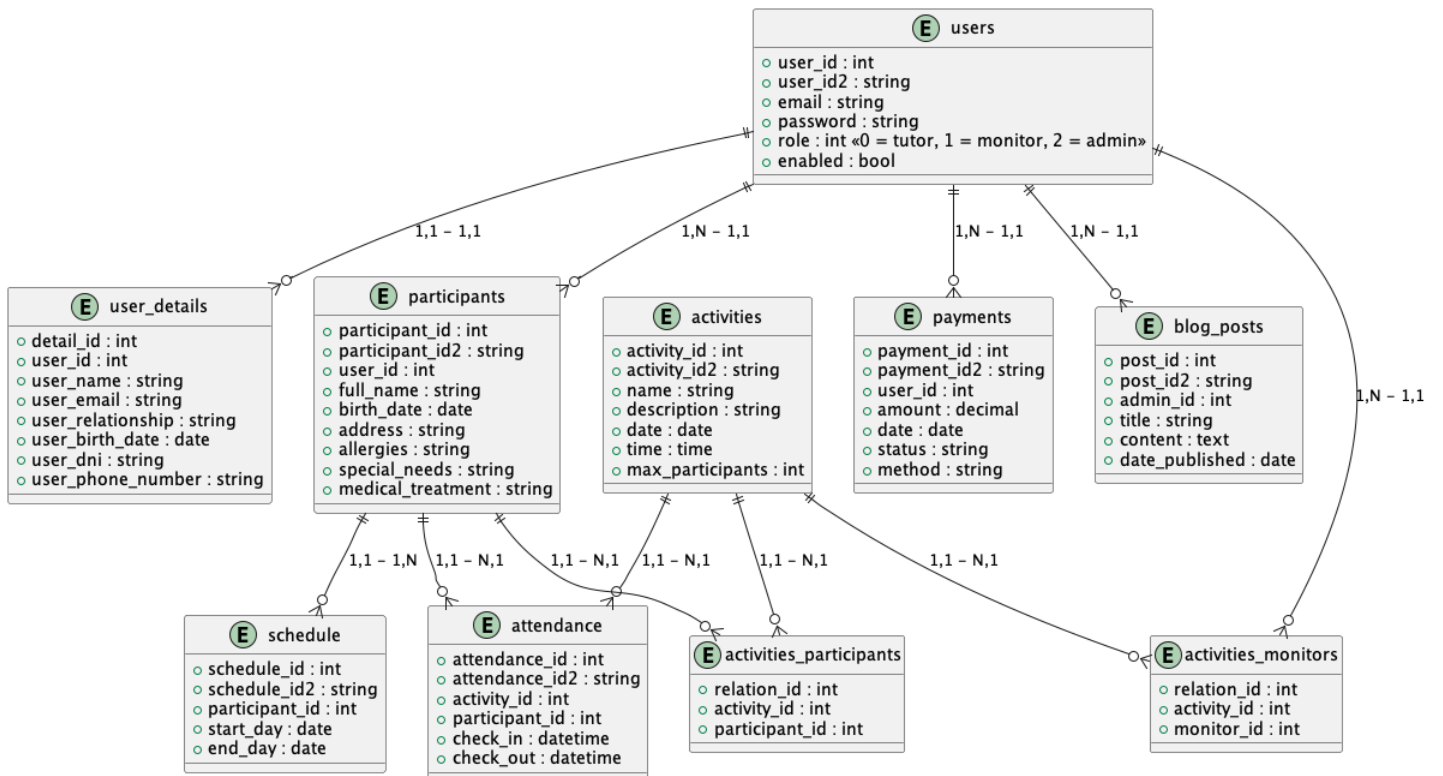


Diagrama de entidad-relación

El **diagrama entidad-relación** ha sido desarrollado para representar las entidades principales del sistema, sus atributos y las relaciones entre ellas. Este modelo permite visualizar cómo los datos están estructurados y cómo las entidades interactúan entre sí, facilitando la organización y gestión de la información en la base de datos.





Anexos

Configuración general de Apache para el proyecto

Para desplegar correctamente el proyecto en el servidor de producción es necesario realizar numerosas configuraciones en diferentes servicios. Entre los más importantes se encuentra Apache, ya que es el servidor web que será utilizado durante el desarrollo de la aplicación.

La configuración general de Apache (la del propio servidor) no ha sido realmente modificada, ya que, en el caso de este proyecto, han sido usados archivos .htaccess. Por otra parte, han sido insertados varios .htaccess para manejar la autenticación de los usuarios en apartados privados (monitores y tutores) dentro del sistema.

En el caso de este proyecto, la configuración general de Apache en .htaccess ha sido la siguiente:

```
src > .htaccess
1  # No mostramos los ficheros
2  Options -Indexes
3
4  # Determinamos que el fichero principal es el index.php
5  DirectoryIndex index.php
6
7  # Configuramos que siempre que entremos en esta carpeta, siempre se ejecute el index.php
8  # Además, en el caso de que se intenten capturar archivos físicos o directorios, estarán disponibles
9  RewriteEngine On
10 RewriteCond %{REQUEST_FILENAME} !-f
11 RewriteCond %{REQUEST_FILENAME} !-d
12 RewriteRule ^.*$ index.php [L,QSA]
13
14 # Página personalizada para errores 404
15 ErrorDocument 404 /apache/errors/404.html
16
17 # Página personalizada para errores 403
18 ErrorDocument 403 /apache/errors/403.html
19
20 # Página personalizada para errores 500
21 ErrorDocument 500 /apache/errors/500.html
```

Options -Indexes: En el caso de que no exista el archivo DirectoryIndex, no se mostrarán todos los directorios del proyecto.

DirectoryIndex: Dentro del funcionamiento del proyecto, es necesario utilizar un archivo `index.php` para manejar todas las rutas y el propio funcionamiento del entorno, por lo que, para que funcione correctamente, es necesario hacer que este archivo se ejecute siempre.

RewriteConditions: Estas directrices indican que se debe de ejecutar siempre el `index.php` y que, en el caso de que se quiera acceder a algún directorio o archivo físico (por ejemplo, un archivo CSS), este sea accesible.

En caso de que no estuvieran estas directrices, al intentar enlazar un CSS o un JS a un archivo HTML, debe de ser procesado primero por el `index.php`

ErrorDocument: Estas directrices indican que si un error 404, 403 o 500, se deben de imprimir unos HTMLs específicos en pantalla.

Configuración de autenticación Digest:

En los apartados de Monitor y Tutor, han sido creados dos archivos `.htaccess` para limitar el acceso a aquellos que tengan unas credenciales.

Para hacer esto, es necesario crear un archivo con las credenciales de los usuarios con el siguiente comando:

```
root@0a2fa606305e:/var/www/html# htdigest -c apache/.htdigest_monitors "Secured Area - Monitors" monitor_1
Adding password for monitor_1 in realm Secured Area - Monitors.
New password:
Re-type new password:
root@0a2fa606305e:/var/www/html# htdigest -c apache/.htdigest_tutors "Secured Area - Tutors" tutor_1
Adding password for tutor_1 in realm Secured Area - Tutors.
New password:
Re-type new password:
root@0a2fa606305e:/var/www/html#
```

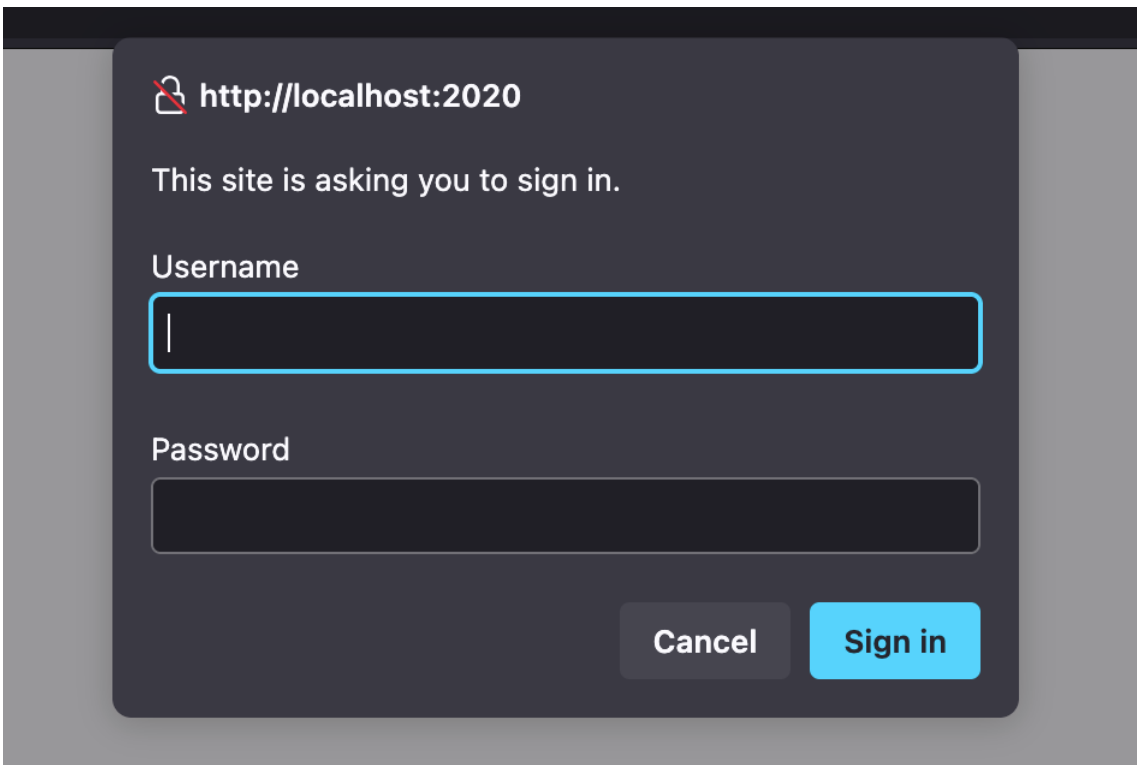
El resultado de este comando es que se crea un fichero `.htdigest` con las credenciales hasheadas.

```
src > apache > .htdigest_monitors
1 monitor_1:Secured Area - Monitors:184120b0ed29d296e9927023912c18b5
2 |
```

Después de ejecutar este comando, es necesario crear un archivo `.htaccess` en el directorio que queremos bloquear.

```
src > pages > TutorEdit >  .htaccess
1  AuthType Digest
2  AuthName "Secured Area - Tutors"
3  AuthDigestProvider file
4  AuthUserFile /var/www/html/apache/.htdigest_tutors
5  Require valid-user
```

Una vez insertada esta configuración, al abrir la página donde esté este `.htaccess`, nos pedirá insertar unas credenciales.



 http://localhost:2020

This site is asking you to sign in.

Username

Password

Cancel Sign in

Una vez hechas todas estas configuraciones, ya se habrá configurado correctamente la autenticación por Digest.